

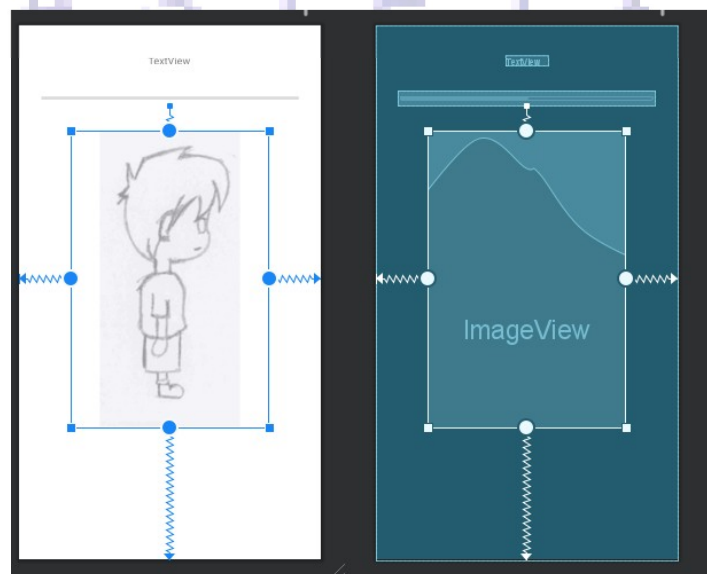
Asynctask Personalizado

Continuando con el proyecto tal cual quedó en el manual del Toast personalizado habrá que crear una nueva actividad de nombre EjemploAsynctask (click derecho en app new->Activity->EmptyActivity). La idea en este caso será la de crear una ventana que muestre una barra de carga, con un texto de porcentajes y una animación de un niño andando. Dicha animación y el resto del progreso serán controlados por una clase que heredaré de AsyncTask para que su ejecución sea paralela al resto de la aplicación.

Para crear los frames de la animación se comenzará por crear un nuevo archivo XML de nombre frames en la carpeta values (click derecho en res->values y seleccionar 'New resource file'):

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string-array name="cursos">
        <item>DAM 1</item>
        <item>DAM 2</item>
        <item>ASIR 1</item>
        <item>ASIR 2</item>
    </string-array>
    <string-array name="imagenes">
        <item>@drawable/andando1</item>
        <item>@drawable/andando2</item>
        <item>@drawable/andando3</item>
        <item>@drawable/andando4</item>
    </string-array>
</resources>
```

Para la nueva actividad que se acaba de crear se seleccionará un ConstraintLayout que contendrá un TextView para contener un porcentaje, una progress bar horizontal y una imagen, sobre la que se irán cargando las distintas imágenes que componen la animación:



El código quedará así:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".EjemploAsyncTask">

    <TextView
        android:id="@+id/tvAnimacion"
        android:layout_width="58dp"
        android:layout_height="15dp"
        android:layout_marginTop="40dp"
        android:text="TextView"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
    />

    <ProgressBar
        android:id="@+id/pbAnimacionr"
        style="?android:attr/progressBarStyleHorizontal"
        android:layout_width="351dp"
        android:layout_height="20dp"
        android:layout_marginTop="36dp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/tvAnimacion"
    />

    <ImageView
        android:id="@+id/ivAnimacion"
        android:layout_width="269dp"
        android:layout_height="405dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/pbAnimacionr"
        app:layout_constraintVertical_bias="0.158"
        app:srcCompat="@drawable/andando1" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

En el archivo java de la actividad habrá que crear 3 atributos privados y sus getters y setters. Además en la función onCreate:

- Habrá que conseguir del archivo XML creado con anterioridad las imágenes que componen la animación. Si no se especifica, se cargaría un array de Strings, pero se necesita un TypedArray, de ahí el uso de la función obtainTypedArray.
- Se obtienen las referencias de la barra de carga, del ImageView y del TextView.
- Se configura la barra de carga.
- Se crea una clase de nombre ProgressAndando, que debe heredar de la clase AsyncTask y debe implementar los siguientes métodos heredados:
 - doInBackground: es la parte que se ejecuta en segundo plano en el hilo.
 - onPreExecute: se ejecuta antes de lanzar el hilo.
 - onPostExecute: se ejecuta una vez terminado el hilo.
 - onProgressUpdate: se utiliza para mostrar al usuario el progreso de lo que se está haciendo en la función doInBackground.

```
public class EjemploAsyncTask extends AppCompatActivity {

    //Typed Array es un array de recursos.
    private TypedArray imagenes = null;
    private ProgressBar barra = null;
    private ImageView imagenCentral = null;
    private TextView texto = null;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_ejemplo_async_task);

        //Obtener los datos del XML Parametro: R.array porque
        //es un array de Strings, que se convierte en un
        //typed array.
        imagenes =
        getResources().obtainTypedArray(R.array.imagenes);

        barra = (ProgressBar) findViewById(R.id.pbAnimacionr);
        imagenCentral = (ImageView)
        findViewById(R.id.ivAnimacion);
        texto = (TextView) findViewById(R.id.tvAnimacion);

        //Parámetros para el ProgressBar
        //Valor máximo
        barra.setMax(100);

        //Inicialización
        barra.setProgress(0);

        //Color de fondo
        barra.setBackgroundColor(Color.GRAY);

        ProgressAndando hilo = new ProgressAndando(this);
```

```
//Ejecución del hilo
ThreadExecutor executor = new ThreadExecutor();
executor.execute(hilo);

}

public void finalizar()
{
    this.finish();
}

public TypedArray getImagenes() {
    return imagenes;
}

public void setImagenes(TypedArray imagenes) {
    this.imagenes = imagenes;
}

public ProgressBar getBarra() {
    return barra;
}

public void setBarra(ProgressBar barra) {
    this.barra = barra;
}

public ImageView getImagenCentral() {
    return imagenCentral;
}

public void setImagenCentral(ImageView imagenCentral) {
    this.imagenCentral = imagenCentral;
}

public TextView getTexto() {
    return texto;
}

public void setTexto(TextView texto) {
    this.texto = texto;
}
}
```

En la clase `ProgressAndando` se implementa la interfaz `Runnable`, que permitirá lanzar aquello que se escriba en la función `run` en segundo plano:

```
public class ProgressAndando implements Runnable
{
    private EjemploAsyncTask actividad = null;

    public ProgressAndando(EjemploAsyncTask actividad)
    {
        this.actividad = actividad;
    }

    @Override
    public void run()
    {
        int numFoto = 0;
        ProgressBar barra = actividad.getBarra();
        ImageView foto = actividad.getImagenCentral();
        TextView texto = actividad.getTexto();
        TypedArray imagenes = actividad.getImagenes();
        Log.d("ander", imagenes.getIndexCount() + "");
        //Inicializar los elementos gráficos
        barra.setProgress(0);
        texto.setText("0%");

        for(int i = 0; i < 100; i++)
        {
            try
            {
                Thread.sleep(100);
                if(numFoto >= 3)
                {
                    numFoto = 1;
                } else
                {
                    numFoto++;
                }
                barra.setProgress(i);

                foto.setImageResource(imagenes.getResourceId(numFoto, -1));
                texto.setText(i + "%");
            } catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        }

        Toast.makeText(actividad, "Tarea finalizada",
        Toast.LENGTH_SHORT).show();
        actividad.finalizar();
    }
}
```

Para poder ejecutar un Runnable en segundo plano será necesaria una clase Executor, que en nuestro caso simplemente ejecuta el comando enviado, pero que puede ser una clase más compleja en la que implementar colas de hilos, detección de eventos en los hilos, captura de resultados y otras operaciones:

```
public class ThreadExecutor implements Executor
{
    @Override
    public void execute(Runnable command)
    {
        new Thread(command).start();
    }
}
```

Para lanzar la nueva actividad desde MainActivity.java:

```
ImageButton btnAsync = (ImageButton)
findViewById(R.id.btnAsync);
btnAsync.setOnClickListener(new View.OnClickListener()
{
    @Override
    public void onClick(View v)
    {
        Intent intent = new Intent(getApplicationContext(),
EjemploAsyntask.class);
        startActivity(intent);
    }
});
```

A continuación, con la intención de aprender a pasar información de una actividad a otra se utilizará una clase Bundle. En el click del botón Async se crea un Bundle al que, a través del método putString, se le introduce un String con clave "saludo". Una vez hecho esto se introduce el Bundle en el intent y se lanza la actividad:

```
btnAsync.setOnClickListener(new View.OnClickListener()
{
    @Override
    public void onClick(View v)
    {
        //Creación de un Bundle para pasar datos a la nueva
        activity
        Bundle bundle = new Bundle();
        bundle.putString("saludo", "Hola amigo@");
        intent.putExtras(bundle);
        startActivity(intent);
    }
});
```

```
}  
});
```

Para leer del Bundle en la actividad nueva:

```
//Leer el bundle  
Bundle bundleRecibido = (Bundle) getIntent().getExtras();  
if(!bundleRecibido.isEmpty())  
{  
    Toast.makeText(getApplicationContext(),  
bundleRecibido.getString("saludo"),  
Toast.LENGTH_SHORT).show();  
}
```